Rooftop Rain Pump Automation — System Architecture

1. Overview

A local controller service running on an Ubuntu server automates rooftop drain pumps that are currently switched manually through SmartThings and Smart Life. The service polls a weather forecast, decides when pumps should run, and switches them — preferring **local LAN control** (Tuya local protocol) with the **SmartThings cloud API as fallback**. Phase 2 adds a physical rain sensor over MQTT for precise, forecast-independent triggering.

2. System Architecture Diagram

```
flowchart TB

    subgraph Cloud["Cloud (Internet)"]

        WX["Weather APIs<br/>Open-Meteo (primary)<br/>NWS api.weather.gov (secondary)"]

        ST["SmartThings Cloud API<br/>(fallback control + state verify)"]

        TUYACLOUD["Tuya Cloud<br/>(one-time: obtain device local keys)"]

    end

    subgraph Server["Ubuntu Server (Docker Compose)"]

        subgraph CTRL["pump-controller (Python)"]

            SCHED["Scheduler<br/>(APScheduler)"]

            FC["Forecast Ingestor"]

            RULES["Rules Engine<br/>(decision logic)"]

            DEV["Device Adapter Layer"]

            TA["Tuya Local Adapter<br/>(tinytuya)"]

            STA["SmartThings Adapter<br/>(REST)"]
```

```
        SAFE["Safety Monitor<br/>(max runtime, cooldown, watchdog)"]
    end
    API["Web UI + REST API<br/>(FastAPI)"]
    DB[("SQLite<br/>state, run history, forecasts")]
    MQTT["Mosquitto MQTT Broker<br/>(Phase 2)"]
    NOTIF["Notifier<br/>(ntfy / email)"]
end
subgraph Roof["Rooftop"]
    P1["Pump 1<br/>(Tuya smart switch/plug)"]
    P2["Pump 2..N"]
    RS["Rain Sensor — Phase 2<br/>ESP32 + tipping-bucket gauge<br/>(ESPHome)"]
end
USER["Browser / Phone<br/>(dashboard, manual override)"]
FC -->|poll hourly forecast| WX
SCHED --> FC
SCHED --> RULES
FC --> DB
RULES --> DEV
RULES --> DB
DEV --> TA
DEV -->|fallback / verify| STA
```

```
STA --> ST

TA -->|LAN, local key| P1

TA -->|LAN| P2

TUYACLOUD -.->|setup only| TA

RS -->|rain rate / tips| MQTT

MQTT -->|Phase 2 input| RULES

SAFE --> DEV

API --> RULES

API --> DB

USER --> API

RULES --> NOTIF
```

3. Functional Description

Forecast Ingestor. Polls Open-Meteo (free, no API key) every 30–60 min for hourly precipitation probability and expected rainfall (mm) at the building's coordinates. NWS is a secondary source for cross-checking (US only). Forecasts are stored in SQLite for the rules engine and dashboard history.

Rules Engine. Runs on a schedule (e.g., every 10 min) and on events. Core logic:

- *Pre-emptive start:* if forecast shows precipitation probability ≥ threshold (e.g., 70%) AND expected accumulation ≥ threshold (e.g., 2 mm) within the lookahead window (e.g., 2 h) → start pumps.
- *Continue:* while rain is forecast/ongoing, keep pumps running with a duty cycle if configured (e.g., 10 min on / 20 min off) to avoid dry running.
- *Post-rain drain:* after rain ends, run pumps for a configurable drain-down period (e.g., 30 min), then stop.
- *Stop:* no rain forecast in window and drain-down complete → pumps off.
- Phase 2: actual rain-rate from the sensor overrides forecast (sensor says raining → run; sensor dry for N minutes after forecast said rain → stop early). Forecast remains for pre-emptive starts.

Device Adapter Layer. Abstracts pump control behind a single interface (`turn_on / turn_off / get_state`). Primary path is **tinytuya** over the LAN (fully local — meets the local-server goal; requires one-time extraction of each device's local key via the Tuya IoT platform). Fallback path is the **SmartThings REST API** (cloud) used when local control fails, and for independent state verification. Every command is verified by reading device state back; mismatches are retried then alerted.

Safety Monitor. Enforces max continuous runtime per pump (dry-run protection), minimum off/cooldown time, and a watchdog: if the service crashes or loses weather data, pumps are commanded off (or left in last safe state) and an alert is sent. Manual override (on/off/auto per pump) from the dashboard always wins over automation, with an auto-revert timeout.

Web UI / REST API. FastAPI serves a simple dashboard: pump states, current mode (auto/manual), forecast summary, run history, rain sensor readings (Phase 2), and override buttons. Same endpoints usable via curl/Home Assistant later.

Notifier. Pushes events (pumps started/stopped, control failure, watchdog trip) via ntfy (free push to phone) or SMTP.

Phase 2 — Rain Sensor. An ESP32 running ESPHome with a tipping-bucket rain gauge publishes rain tips/rate to Mosquitto on the server. The rules engine subscribes and blends sensor truth with forecast prediction. No cloud involved.

4. Components

Software (all on the Ubuntu server, Docker Compose)

Component	Technology	Role
pump-controller	Python 3.12, APScheduler, tinytuya, httpx, paho-mqtt	Forecast polling, rules engine, device control, safety
Web UI / API	FastAPI + Uvicorn, Jinja2 or small SPA	Dashboard, manual override, REST API
Database	SQLite (via SQLAlchemy)	Config state, run history, forecast cache, event log
MQTT broker	Eclipse Mosquitto (container)	Rain sensor ingestion (Phase 2)
Notifications	ntfy or SMTP	Alerts and status pushes

Component	Technology	Role
Weather sources	Open-Meteo API (primary), NWS API (secondary)	Hourly precipitation forecast; both free, no key needed
Setup tooling	tinytuya wizard + Tuya IoT platform account	One-time local-key extraction for each pump switch
Deployment	Docker Compose, systemd (compose up on boot)	Reliability, restarts, logs

Hardware

Item	Notes
Existing Tuya/Smart Life smart switches or plugs driving the pumps	Reused as-is; controlled locally via LAN. Must be on the same LAN/VLAN as the server with fixed IPs (DHCP reservations).
Ubuntu server	Anything always-on (mini PC, NUC, Pi 4/5 also fine). Wired Ethernet recommended.
Phase 2: ESP32 dev board (e.g., ESP32-WROOM)	Runs ESPHome; Wi-Fi to LAN, publishes to MQTT.
Phase 2: Tipping-bucket rain gauge (e.g., Misol WH-SP-RG, ~$15–25)	Reed-switch pulse output → ESP32 GPIO; gives actual rain rate.
Phase 2 (optional): Float/level switch in each collection basin	Wired to the ESP32; the most direct "water present" signal and best dry-run protection.
Optional: weatherproof enclosure, 5 V supply for ESP32	Rooftop mounting.

5. Key Design Decisions

- **Local-first control:** tinytuya keeps pump switching on the LAN and working during internet outages (forecast unavailable, but sensor + manual + last-known rules still work). SmartThings cloud is fallback only.
- **Forecast now, sensor later:** the rules engine takes an abstract "rain signal" input, so adding the MQTT sensor in Phase 2 is a config change plus one new signal source — no redesign.
- **Fail-safe defaults:** on any uncertainty (no data, control failure), pumps default off with an alert; max-runtime caps prevent dry running.

- **SQLite over Postgres:** single-host, low write volume; zero admin.